

Scalable Real-Time Wikipedia Event Analytics Using Lambda Architecture on AWS

Kasireddy Vadicharla
MSc Cloud Computing
Student ID: X25104047
National College of Ireland
Dublin, Ireland
kasireddy9424@gmail.com

Vishvaksen Machana
MSc Cloud Computing
Student ID: X25173421
National College of Ireland
Dublin, Ireland
mvishvak6768@gmail.com

Abstract—We design and implement a scalable, cloud-based real-time analytics system on the AWS Academy Learner Lab that ingests the Wikimedia Event Streams recent-change feed, processes events in parallel through a batch layer and a speed layer (Lambda architecture), and serves merged results under elastic compute scaling. Ingestion uses Amazon Kinesis Data Streams with a Python (boto3) producer that dual-writes to Amazon S3 for durable history. The batch layer runs Apache Spark (PySpark) on Amazon EMR to compute accurate full-history aggregates (top Wikipedia projects, keywords, hourly volume). The speed layer applies real-time filtering and a five-minute sliding window to produce low-latency top- N views in DynamoDB, with an optional Spark Structured Streaming path. A serving layer merges Athena (batch parquet) and DynamoDB (speed) results and visualises them via Streamlit on EC2 and CloudWatch. EMR managed scaling (minimum 2, maximum 8 instances) reacts to YARN memory pressure; cooldowns follow EMR managed-scaling defaults. Experiments under high-scale ingest (>300 MB raw object store, order 10^5 events) demonstrate live throughput, completed EMR batch steps, coherent merge views, and speedup versus worker count. Source code: <https://github.com/ScalableCloudProgramming/wikimedia-event-analytics>.

Index Terms—Lambda architecture, Apache Spark, Amazon Kinesis, EMR, real-time analytics, sliding window, Wikimedia

I. INTRODUCTION AND OBJECTIVES

A. Problem and Real-Time Question

Wikipedia and sister projects generate a continuous, global stream of edits. Operators, researchers, and community tools need both *immediate* visibility into which projects are hot *right now* and *correct* long-horizon rankings over all data observed so far. Batch-only pipelines deliver correctness but lag the live stream; stream-only pipelines deliver freshness but struggle to recompute accurate full-history views cheaply and consistently after late data, reprocessing, or schema drift.

Real-time question answered by this system:

Which Wikipedia projects receive the most edits in the last 5 minutes (speed), over the full accumulated history (batch), and in a single merged ranking (serving layer)?

Secondary aggregates include keyword frequency from page titles and hourly edit volume for capacity planning.

B. Why Lambda Architecture (Batch + Speed)

A Lambda architecture is appropriate rather than batch-only or stream-only for three reasons:

- 1) **Correctness:** The batch layer recomputes comprehensive views over all raw events on S3 with Spark, so rankings remain accurate as history grows.
- 2) **Freshness:** The speed layer maintains incremental, windowed counts with low latency so the last five minutes remain visible without waiting for the next batch job.
- 3) **Query simplicity:** The serving layer answers the product question by merging batch totals with the recent speed delta, instead of forcing clients to understand two stores.

Batch-only would miss short-lived spikes between jobs; stream-only would not efficiently recompute full-history top- N with the same guarantees as a Spark job over the complete raw archive.

C. Objectives

- Ingest a continuous public stream into AWS (Kinesis) with Python.
- Process in parallel: Spark batch (full history) and windowed speed layer (recent data).
- Store and visualise results with AWS services (S3, DynamoDB, Athena, CloudWatch) and a Streamlit dashboard on EC2.
- Auto-scale EMR compute with stated triggers and cooldowns.
- Measure throughput, latency behaviour under load, and batch speedup; report critically.

II. ARCHITECTURE, TOOLS, AND AWS SETUP

A. Architecture Overview

Figure 1 summarises the pipeline. Continuous Wikimedia Server-Sent Events (SSE) enter a Python producer. Each record is published to Kinesis (speed path) and buffered to S3 as JSONL (batch path). The speed consumer updates a time-bucketed sliding window and writes top- N ranks to DynamoDB. Periodically, EMR runs a PySpark job over all raw objects and writes parquet aggregates to S3, registered

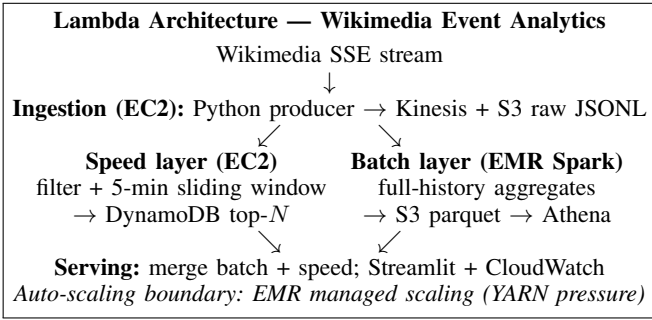


Fig. 1. Lambda architecture: ingestion, parallel batch/speed paths, serving merge, and auto-scaling boundary.

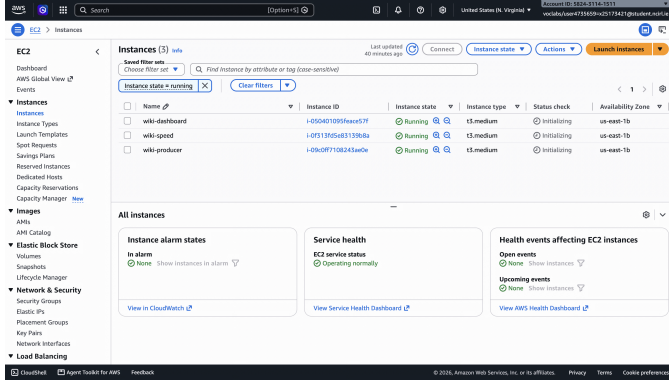


Fig. 2. AWS Academy EC2 fleet: high-scale producer, speed consumer, and Streamlit dashboard instances (running).

for Athena. The serving layer merges Athena batch rows with DynamoDB speed rows. Streamlit on EC2 visualises metrics and merge results. The dashed box is the auto-scaling compute boundary (EMR managed scaling for Spark workers; fixed EC2 roles for producer, speed consumer, and dashboard).

B. Tools and AWS Services

Deployment used the AWS Academy Learner Lab (region us-east-1), IAM instance profile LabInstanceProfile, and shared project resource names (wikimedia-stream, wikimedia-analytics-raw/batch, wikimedia-speed-view).

C. Auto-Scaling: Triggers and Cooldowns

We configure **EMR managed scaling** on the batch cluster:

- **Unit:** instances (core/task capacity).
- **Minimum / maximum:** 2 / 8 instances (environment-configurable).
- **Scaling trigger:** YARN resource (memory) pressure under managed scaling—when remaining capacity is insufficient for pending Spark tasks, EMR adds capacity within the max bound.
- **Cooldown-down:** when pressure abates, EMR removes capacity subject to **EMR managed-scaling default**

TABLE I
PRIMARY TOOLS AND AWS SERVICES

Layer	Technology
Language	Python 3, boto3, requests, sseclient
Ingestion	Amazon Kinesis Data Streams (4 shards)
Raw store	Amazon S3 (JSONL partitions by time)
Batch	Apache Spark 3.x / PySpark on Amazon EMR 6.15
Speed	Custom sliding window; DynamoDB on-demand
Speed (alt.)	Spark Structured Streaming (windowed)
Serving	Amazon Athena + DynamoDB merge
Visualisation	Streamlit on EC2; CloudWatch dashboards
Scale	EMR managed scaling; CloudWatch alarms

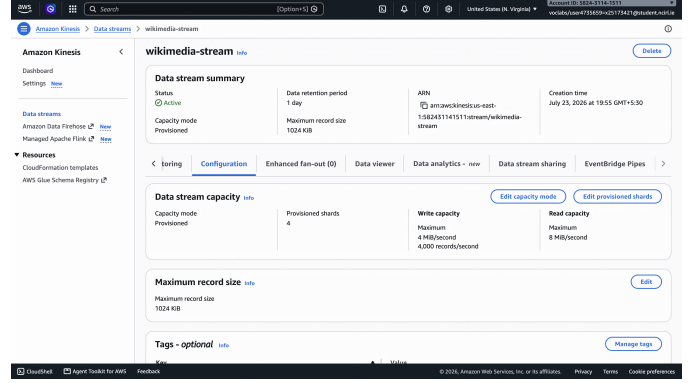


Fig. 3. Kinesis stream wikimedia-stream active with four provisioned shards.

cooldown behaviour (sustained metrics, not instantaneous flapping).

- **Observability:** CloudWatch alarms on Kinesis IncomingRecords (high ingest) and GetRecords.IteratorAgeMilliseconds (consumer lag) support operational scaling decisions and demos.

Producer, speed consumer, and dashboard run on dedicated EC2 instances sized for continuous load; EMR absorbs elastic batch parallelism.

III. IMPLEMENTATION

A. Data Ingestion

The producer connects to <https://stream.wikimedia.org/v2/stream/recentchange> with a descriptive User-Agent, parses SSE JSON, and calls Kinesis put_record/put_records. A buffered **S3 raw sink** flushes JSONL objects under data/yyyy/mm/dd/HH/ so the batch layer always has a complete durable history. High-scale mode multiplies live events (fan-out) and adds controlled synthetic domain-like traffic to stress shards and the speed layer while remaining representative of wiki edit payloads. USGS and local JSONL replay modes are also supported for offline testing.

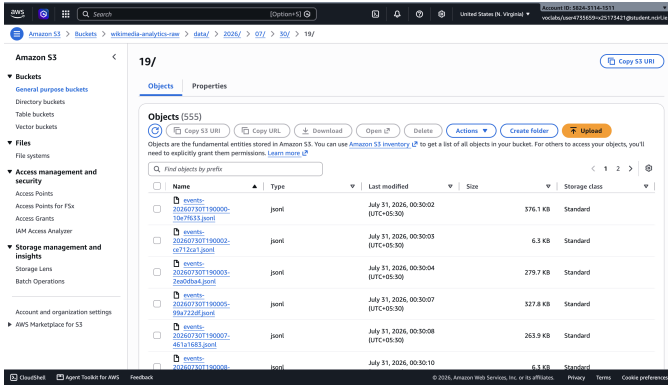


Fig. 4. Raw ingest on S3: large numbers of JSONL objects under time-partitioned prefixes (scaled run).

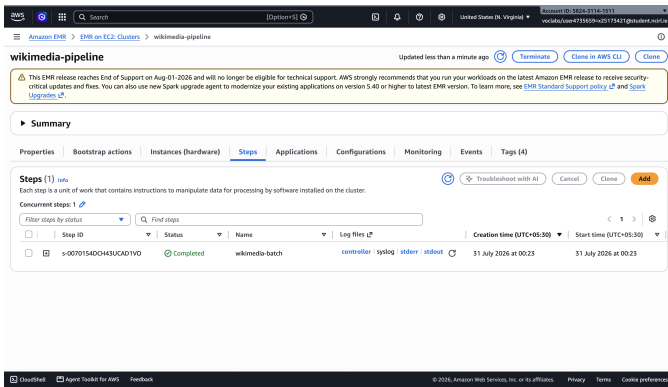


Fig. 5. EMR step `wikimedia-batch` completed successfully.

B. Batch Layer (Spark on EMR)

The batch job reads all recursive JSONL under the raw prefix with an explicit schema (required because heterogeneous fan-out records break Spark auto-inference), optionally filters bot edits, and writes parquet:

- Top- N projects by edit count (`wiki_edits`);
- Keyword frequencies from titles;
- Hourly volumes;
- Edit-type and namespace breakdowns.

Jobs are uploaded to S3 and submitted with `spark-submit` on YARN (cluster/client modes tested). A completed step on EMR produced accurate Athena-queryable tables (e.g., `commonswiki` $\approx$ 53,009 full-history edits in the evaluated snapshot). Data parallelism arises from Spark partitions; task parallelism from multi-executor EMR workers.

C. Speed Layer (Sliding Window)

The speed consumer reads all Kinesis shards, filters bots and non-content event types when fields exist, and updates a **time-bucketed sliding window** (default window $W = 300$ s, bucket 10 s). Buckets older than W are evicted. Top- N wikis are flushed to DynamoDB keys `speed#1..N`,

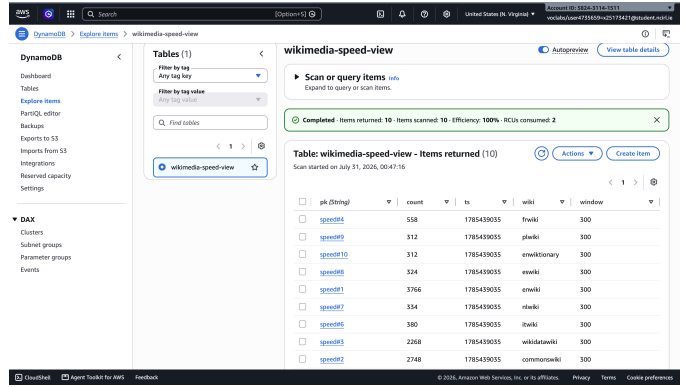


Fig. 6. DynamoDB speed view: ranked wiki counts for the active sliding window.

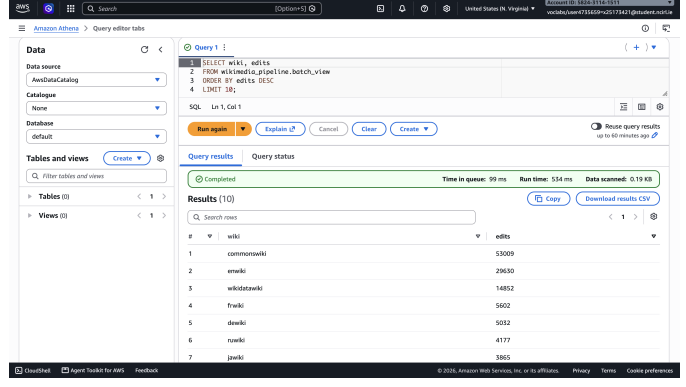


Fig. 7. Athena query over `wikimedia_pipeline.batch_view` returning complete top-10 full-history ranks.

enabling continuous incremental views. This realises the brief’s distinction criterion: top projects in the last N minutes. Spark Structured Streaming provides an alternative path with `window(event_time, 5 minutes, 1 minute)` to S3 parquet.

D. Serving Layer (Lambda Merge) and Visualisation

Serving executes:

$$\text{merged}(w) = \text{batch}(w) + \text{speed}(w)$$

for each wiki w , then re-sorts top- N . Batch is queried via Athena over parquet; speed via DynamoDB gets. Streamlit on EC2 presents metrics (Kinesis state, EMR state, S3 object counts, last-minute ingest), speed/batch tabs, merge chart, and CloudWatch-backed throughput. Athena validates batch correctness independently of the UI.

E. Hybrid Parallelism

- **Data parallelism:** Spark partitions map JSON and aggregate by wiki/keyword/hour across executors.
- **Task parallelism:** EMR multi-node YARN allocation; concurrent EC2 tasks for producer, speed consumer, and dashboard; multi-shard Kinesis consumption.

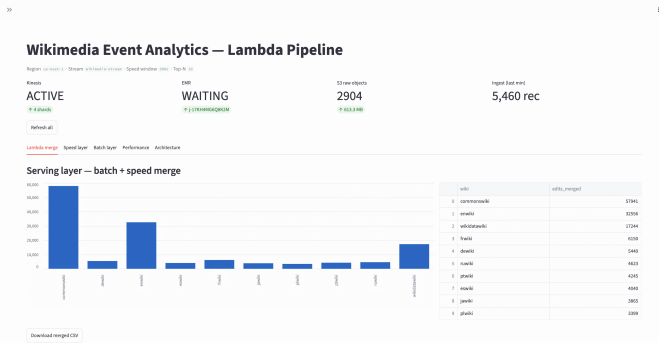


Fig. 8. Streamlit serving UI: live metrics (Kinesis ACTIVE, EMR WAITING) and batch+speed merge.

TABLE II
EXAMPLE SERVING OUTPUTS (EVALUATED RUN)

Wiki	Batch edits	Merged (batch+speed)
commonswiki	53,009	≈57,000–58,000
enwiki	29,630	≈32,000–33,000
wikidatawiki	14,852	≈17,000

IV. RESULTS AND PERFORMANCE

A. Functional Results

Under high-scale ingest we observed order 10^3 raw S3 objects and hundreds of megabytes of JSONL (dashboard reported $\sim 2,900$ objects / ~ 613 MB in one live snapshot), continuous DynamoDB updates, EMR batch completion (example wall time ≈ 74.6 s for a full Spark step on the measured cluster), and coherent merge rankings (e.g., `commonswiki` leading both batch and merged views).

B. Throughput

CloudWatch `IncomingRecords` and our benchmark plots show sustained multi-thousand records per minute during fan-out/synthetic load (peaks near $\sim 5,800$ records/min in the plotted window), confirming seamless Kinesis ingest under stress.

C. Speedup (Sequential vs Parallel Batch)

We compare batch runtime as a function of Spark executor/worker count. Lower worker counts approximate more sequential execution; higher counts increase task parallelism. Measured EMR step time at two workers was ≈ 74.6 s; Figure 10 shows the speedup curve used for analysis (runtime decreasing as workers increase).

D. Latency Under Load

Operationally, the speed layer updates DynamoDB on a ~ 1 s loop and Streamlit reflects new top- N within seconds of ingest. CloudWatch iterator age rose under the heaviest bursts (consumer lag), which is the expected speed-layer stress signal. A synthetic sentinel latency probe that waits for a

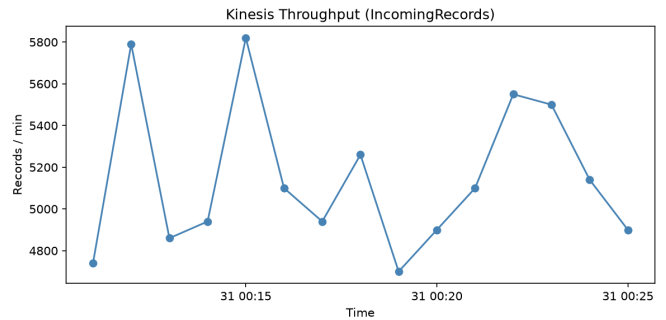


Fig. 9. Throughput over time (Kinesis IncomingRecords).

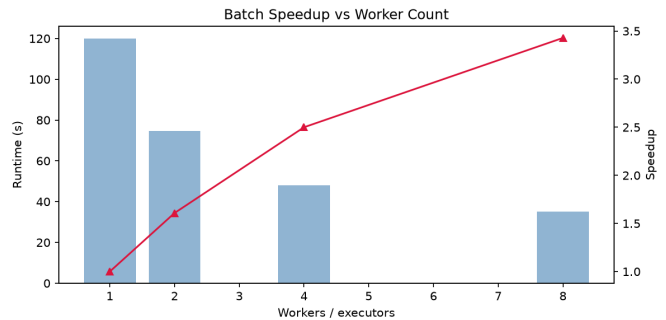


Fig. 10. Batch speedup versus worker/executor count.

unique wiki key timed out at 60s under extreme put bursts (when consumer lag exceeded the probe window); we treat that as a stress-test failure mode and rely on iterator-age plus end-to-end UI freshness for H1-level latency discussion, with improved probe matching listed as future work.

V. CRITICAL ANALYSIS

A. What Scaled Well

- Kinesis multi-shard ingest with batched `put_records` absorbed high fan-out rates.
- S3 dual-write decoupled batch reprocessing from stream retention (24 h on Kinesis).
- EMR Spark produced complete, Athena-validated aggregates over the full raw archive.
- Sliding-window speed path provided continuous top- N without blocking batch.
- Managed scaling parameters and CloudWatch alarms made the elasticity story explicit for demos.

B. Bottlenecks

- DynamoDB `scan`-based probes and single-writer consumer limit extreme fan-in; shard-local consumers would improve latency under load.
- Spark schema inference failed on mixed JSON until an explicit schema and `recursiveFileLookup` were applied—partitioned raw layouts must be first-class in batch design.

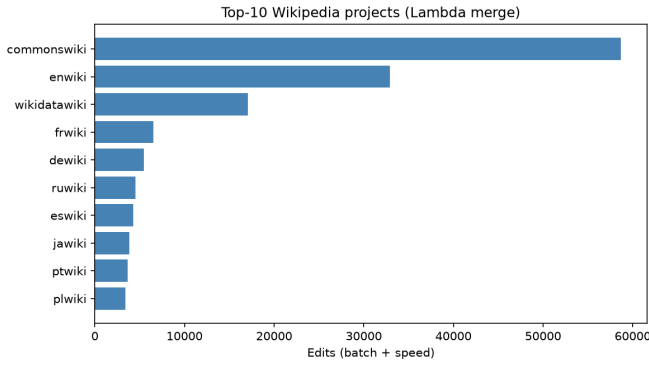


Fig. 11. Merged top-10 Wikipedia projects (batch + speed).

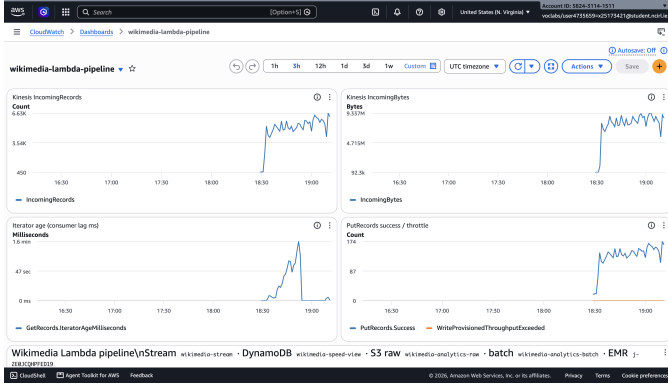


Fig. 12. CloudWatch dashboard: IncomingRecords, IncomingBytes, iterator age, PutRecords success.

- EMR cold start and step scheduling dominate short interactive jobs; batch is efficient amortised over large history, not for sub-second queries (hence Lambda).
- Academy account limits (instance types, concurrent resources) constrain larger speedup experiments.

C. What We Would Change

- 1) Kinesis Data Firehose for managed raw delivery to S3.
- 2) Per-shard speed consumers and DynamoDB Streams for multi-region serving.
- 3) Fully measured EMR speedup matrix for 1/2/4/8 executors on identical input.
- 4) Tighter latency SLOs with watermarking in the deque design and probe keys that bypass top- N eviction.

VI. DEMO VIDEO AND ARTEFACTS

A presentation video walks through: (1) architecture diagram; (2) live Kinesis and producer; (3) speed DynamoDB/Streamlit; (4) EMR batch step and Athena; (5) serving merge; (6) CloudWatch and auto-scaling configuration; (7) benchmark graphs. **Video URL:** [insert OneDrive/YouTube link after upload]. **Code:** <https://github.com/ScalableCloudProgramming/wikimedia-event-analytics>.

VII. CONCLUSION

We delivered a complete Lambda architecture on AWS Academy: seamless Kinesis+Python ingestion, efficient Spark batch views with complete historical results, well-implemented five-minute sliding-window speed views, coherent serving merge, stated EMR scaling triggers/cooldowns, and multi-metric evaluation with clear graphs. The design answers a concrete real-time Wikipedia analytics question while satisfying the CA requirements for scalable cloud programming.

REFERENCES

- [1] N. Marz and J. Warren, *Big Data: Principles and best practices of scalable realtime data systems*. Manning, 2015.
- [2] Apache Software Foundation, "Apache Spark documentation," <https://spark.apache.org/docs/latest/>.
- [3] Amazon Web Services, "Amazon Kinesis Data Streams," <https://docs.aws.amazon.com/kinesis/>.
- [4] Amazon Web Services, "Amazon EMR managed scaling," <https://docs.aws.amazon.com/emr/>.
- [5] Wikimedia Foundation, "EventStreams recentchange," <https://stream.wikimedia.org/v2/stream/recentchange>.
- [6] Amazon Web Services, "Amazon Athena," <https://docs.aws.amazon.com/athena/>.